



Politechnika
Wrocławska

Rozpoznawanie i przetwarzanie obrazów - Projekt

Rozpoznawanie budynków kampusu PWr na podstawie fragmentów elewacji - Dokumentacja końcowa

Mikosiński Kacper

NR INDEKSU: 280985

Oleksandr Nedosiek

NR INDEKSU: 275978

Prowadzący:

mgr inż. Tomasz Serafin

Wrocław, 2026

Streszczenie

Niniejsze opracowanie podsumowuje projekt inżynierski, którego celem było zaprojektowanie, implementacja oraz walidacja rozproszonego systemu wspomagającego orientację przestrzenną na kampusie głównym Politechniki Wrocławskiej. Rozwiązanie opiera się na analizie obrazów elewacji rejestrowanych na żywo lub pobieranych z galerii urządzeń mobilnych, przetwarzanych asynchronicznie przez środowisko serwerowe.

System składa się z dwóch głównych komponentów:

1. **Klienta mobilnego** (framework Flutter) wyposażonego w intuicyjny interfejs, moduł kompresji multimediów, tryb skanowania na żywo (Live Scanner) oraz funkcję automatycznego wykrywania adresu IP serwera przez rozgłoszenia UDP Broadcast.
2. **Serwera AI** (FastAPI, PyTorch), realizującego potok klasyfikacji w oparciu o autorski model głębokiego uczenia maszynowego.

Potok identyfikacyjny wykorzystuje autorską sieć konwolucyjną (SimpleModel) złożoną z czterech bloków spłotowych z warstwami BatchNorm2d i dropout. Odporność na zmienne warunki oświetleniowe i perspektywę zapewniono poprzez zaawansowaną augmentację danych (w tym Color Jitter). Problem niezbalansowania 10 klas budynków (m.in. A-1, C-13, D-20) zniwelowano ważoną funkcją straty i modyfikatorem uczenia. Dodatkowo, w celu odrzucania obiektów spoza kampusu, wdrożono próg pewności predykcji (Confidence Threshold = 60.0%).

Ewaluacja systemu na odizolowanym, zbalansowanym zbiorze testowym liczącym 1000 fotografii wykazała jego umiarkowaną przydatność w warunkach rzeczywistych, plasując rozwiązanie jako działający prototyp inżynierski o wysokim potencjale rozwojowym. Algorytm osiągnął globalną dokładność klasyfikacji (*Accuracy*) na poziomie 69,0%, precyzję (*Macro Precision*) równą 0,72 oraz czułość (*Macro Recall*) wynoszącą 0,69. Analiza ujawniła wysoką skuteczność w identyfikacji obiektów o unikalnej architekturze (D-20, C-13) przy jednoczesnym zidentyfikowaniu barier strukturalnych (przenikanie cech geometrycznych w parach C-16/C-18 oraz H-6/C-3-4), co wytycza precyzyjne kierunki dalszego rozwoju potoku AI w stronę architektur Vision Transformer (ViT).

Spis treści

1	Wstęp, Cele Projektu i Ewolucja Założeń	4
1.1	Wstęp	4
1.2	Cele projektu	4
1.3	Ewolucja założeń	5
2	Środowisko Badawcze i Architektura Wersji 1.0	7
2.1	Kompletny opis i struktura zbioru danych	7
2.2	Schemat blokowy przetwarzania (Pipeline)	7
2.2.1	Interfejs graficzny aplikacji mobilnej (Warstwa Klienta)	9
2.2.2	Logi systemowe i komunikaty diagnostyczne (Warstwa Serwera)	10
2.3	Opis i uzasadnienie wyboru narzędzi oraz bibliotek	11
3	Metodyka Walidacyjna i Scenariusze Testowe	12
3.1	Założenia ewaluacyjne i środowisko badawcze	12
3.2	Definicja matematyczna metryk ewaluacyjnych	13
3.3	Zdefiniowane scenariusze i fazy testowe	14
3.3.1	Faza 1: Ewaluacja analityczna (Testy skryptowe na zbiorze danych)	14
3.3.2	Faza 2: Testy środowiskowe i aplikacyjne (Testy integracyjne)	14
4	Wyniki Walidacji i Testów Środowiskowych	15
4.1	Wyniki Ewaluacji Analitycznej (Faza 1)	15
4.2	Wyniki Testów Środowiskowych i Integracyjnych (Faza 2)	16
5	Analiza Błędów i Ograniczenia Systemu	19
5.1	Wykaz i Analiza Błędów Typu False Positive oraz False Negative	19
5.2	Studia Ablacyjne i Przyczyny Generowania Błędów	20
5.3	Wąskie Gardła Technologiczne	20
6	Podsumowanie i Wnioski	20
6.1	Ocena Końcowa Przydatności Systemu w Środowisku Rzeczywistym	20
6.2	Kierunki Rozwoju i Analiza Krytyczna (Dodatkowe Pół Roku Pracy)	21

1 Wstęp, Cele Projektu i Ewolucja Założeń

1.1 Wstęp

Współczesny kampus główny Politechniki Wrocławskiej charakteryzuje się wysokim stopniem złożoności architektonicznej, obecnością zróżnicowanych stylów budowlanych oraz dynamiczną strukturą przestrzenną. Dla nowych studentów rozpoczynających naukę, uczestników ogólnokrajowych i międzynarodowych konferencji naukowych, a także gości zewnętrznych, sprawne poruszanie się w terenie i szybkie odnalezienie właściwego gmachu stanowią istotne wyzwanie logistyczne.

Standardowe narzędzia nawigacyjne oparte na globalnych systemach pozycjonowania satelitarne (GPS) wykazują w tym specyficznym obszarze krytyczne ograniczenia. Dodatkowo tradycyjne, statyczne plany sytuacyjne i tablice informacyjne rozmieszczone na wolnym powietrzu wymagają od użytkownika ciągłego, samodzielnego korelowania własnej pozycji geograficznej z otoczeniem, co w stresujących sytuacjach bywa nieintuicyjne i czasochłonne.

Niniejszy projekt dedykowany jest rozwiązaniu powyższego problemu logistycznego poprzez implementację zaawansowanych metod wizyjnej identyfikacji obiektów architektonicznych. Dziedzina ta, stanowiąca jeden z kluczowych fundamentów rozwoju współczesnej wizji komputerowej (ang. *computer vision*), pozwala na automatyczną ekstrakcję unikalnych cech geometrycznych, teksturalnych bezpośrednio z cyfrowego obrazu fasady budynku. Przetwarzanie i klasyfikacja tego typu danych reprezentacji wizualnej za pomocą głębokich konwolucyjnych sieci neuronowych (CNN) umożliwia stworzenie autonomicznego asystenta nawigacyjnego, który do poprawnego działania nie wymaga stabilnego sygnału GPS, a jedynie warstwy wizyjnej rejestrowanej w czasie rzeczywistym przez kamerę powszechnie dostępnego urządzenia mobilnego.

1.2 Cele projektu

Głównym celem projektu sformułowanym na początkowym etapie prac badawczo-rozwojowych (Etap 1) było zaprojektowanie, wytrenowanie i wdrożenie kompletnego, bezpiecznego i rozproszonego systemu typu klient-serwer, zdolnego do automatycznego rozpoznawania wybranych budynków kampusu Politechniki Wrocławskiej na podstawie zdjęć ich fragmentów elewacji wykonanych za pomocą smartfona.

Do szczegółowych celów operacyjnych, zdefiniowanych w pierwotnej deklaracji projektowej, należały:

1. **Zaprojektowanie i budowa własnego zbioru danych (datasetu):** Pozyskanie i strukturyzacja unikalnego zestawu fotografii przedstawiających charakterystyczne elementy architektoniczne (drzwi wejściowe, okna, portale, specyficzne tekstury okładzin ściennych, unikalne przeszklenia i perforacje fasad) dla kluczowych obiektów Politechniki Wrocławskiej, ze szczególnym uwzględnieniem budynków A-1, C-13 oraz D-20. Wstępne, teoretyczne założenia zakładały zgromadzenie zaledwie kilkadziesiąt zdjęć dla każdej z analizowanych klas.
2. **Wytrenowanie modelu głębokiego uczenia maszynowego:** Zaprojektowanie architektury głębokiej sieci neuronowej dedykowanej zadaniu klasyfikacji obrazów, cechującej się wysoką zdolnością generalizacji (ang. *generalization ability*). Model

miał poprawnie identyfikować budynki uwiecznione w skrajnie zróżnicowanych warunkach oświetleniowych, pogodowych oraz przy zmiennej perspektywie i odległości od elewacji.

3. **Implementacja mobilnego interfejsu użytkownika:** Stworzenie ergonomicznej aplikacji mobilnej stanowiącej warstwę prezentacji systemu. Interfejs miał umożliwiać intuicyjne wykonanie zdjęcia wbudowanym modułem aparatu lub pobranie pliku z lokalnej galerii multimediów, a następnie realizować bezpieczną transmisję strumienia bajtów bezpośrednio do serwerowego modułu analizy obrazu.
4. **Zwrotne prezentowanie informacji kontekstowej:** Implementacja mechanizmu powiązania wyjściowej klasy modelu z bazą wiedzy o kampusie. System po stronie serwerowej miał przetworzyć obraz, dokonać wnioskowania (inferencji), a wynik końcowy w postaci oficjalnej nazwy budynku oraz przypisanego do niego wydziału (np. Wydział Informatyki i Telekomunikacji dla budynku C-16) przesyłać zwrotnie za pomocą protokołu sieciowego do aplikacji klienta w celu czytelnej prezentacji użytkownikowi.

1.3 Ewolucja założeń

Rzeczywiste prace implementacyjne, optymalizacyjne oraz testy przeprowadzone w Etapie 2 i Etapie 3 zweryfikowały i zmieniły pierwotne, czysto teoretyczne założenia projektowe. Zderzenie teorii z rzeczywistością inżynierską wymusiło redefinicję strategii zespołu w trzech kluczowych obszarach:

1. **Skala, wolumen i dywersyfikacja zbioru danych:** W Etapie 1 zakładano, że „*dataset będzie zawierał kilkadziesiąt zdjęć dla każdego z analizowanych budynków*”. Wstępne próby treningowe prototypu w Etapie 2 jednoznacznie wykazały, że tak skromna próba przy głębokim podobieństwie materiałów budowlanych na kampusie (np. podobieństwo cegły budynków A-1 czy D-20, powtarzalne aluminiowe profile okienne) prowadzi do katastrofalnego zjawiska przeuczenia sieci (ang. *overfitting*). Model zapamiętywał konkretne kadry zamiast uczyć się unikalnych cech dystynktywnych architektury. W efekcie baza danych ewoluowała w stronę masowej rozbudowy: ostateczny zbiór rozszerzono do **10 klas budynków** (włączając obiekty trudne wizualnie, oparte na powtarzalnej architekturze wielkopłytowej i kontenerowej, jak C-1 czy H-6). Finalny zbiór walidacyjno-testowy powiększono do **6620 fotografii**. Aby zapobiec przeuczeniu autorskiej sieci SimpleModel, w kodzie modułu uczącego (Model.py) wdrożono agresywną, niedeterministyczną augmentację danych w locie. Zastosowano losowe transformacje perspektywiczne (RandomPerspective o skali zniekształcenia 0.3), obroty (RandomRotation do 30°), losowe docinanie i skalowanie (RandomResizedCrop(256)) oraz zaawansowane fluktuacje przestrzeni barwnej (ColorJitter modyfikujący losowo jasność, kontrast i saturację o 0.3). Zmiana ta pozwoliła na zasymulowanie skrajnych zachowań użytkownika końcowego (stojącego pod ostrym kątem, blisko fasady lub w cieniu).
2. **Uproszczenie architektury AI i matematyczna filtracja anomalii:** W Etapie 2 rozważano wielostopniowy potok przetwarzania danych, wprowadzający zewnętrzny model filtrujący (np. oparty na architekturze MobileNet), którego zadaniem miało być odrzucanie obiektów niebędących infrastrukturą kampusu (ludzie,

pojazdy, jedzenie). Dokładna analiza wydajnościowa oraz profilowanie kodu na etapie integracji wykazały jednak, że utrzymywanie dwóch niezależnych modeli AI generuje krytyczne wąskie gardła: gwałtownie wzrosło zużycie pamięci VRAM/RAM serwera, czas odpowiedzi HTTP (*latency*) przekroczył granice akceptowalności dla aplikacji czasu rzeczywistego, a złożoność utrzymania kodu produkcyjnego wzrosła nieproporcjonalnie do zysków. W ostatecznej architekturze 1.0 zrezygnowano z zewnętrznych filtrów na rzecz jednokładowego potoku opartego w całości na dedykowanej sieci splotowej SimpleModel (4 bloki splotowe, normalizacja BatchNorm2d, Dropout 0.5). Odporność na anomalie i zdjęcia spoza kampusu zrealizowano w sposób zoptymalizowany i czysto matematyczny po stronie serwera (`server.py`), implementując sztywny próg pewności predykcji (`CONFIDENCE_THRESHOLD = 60.0`). Jeśli znormalizowane wyjście z funkcji Softmax nie osiągnie poziomu 60% ufności, system automatycznie przerywa egzekucję potoku i zwraca bezpieczną flagę "Nie rozpoznano", co w pełni wyeliminowało potrzebę obciążania architektury pomocniczymi sieciami neuronowymi.

3. **Architektura sieciowa i optymalizacja komunikacji klient-serwer:** Pierwotna wizja systemu zakładała prostą aplikację mobilną wysyłającą statyczne zdjęcia na zahardkodowany, stały adres IP serwera obliczeniowego. Podejście to okazało się niewykonalne w warunkach rzeczywistych z uwagi na mechanizmy dynamicznego przydzielania adresów IP w lokalnych sieciach akademickich i konsumenckich Wi-Fi (protokół DHCP). Wprowadzało to konieczność ciągłej rekompilacji kodu źródłowego klienta przy każdej zmianie konfiguracji sieciowej hosta. W ostatecznej wersji systemu założenie to zastąpiono nowoczesnymi mechanizmami sieciowymi niskiego poziomu. W aplikacji mobilnej Flutter (`main.dart`) zaimplementowano asynchroniczny demon komunikacyjny wysyłający zapytania rozgłoszeniowe za pomocą protokołu UDP Broadcast na port 9000, na które serwer (`server.py`) automatycznie odpowiada swoim aktualnym adresem IP w sieci lokalnej (funkcja Auto-Discovery). Ponadto, zamiast pojedynczych zdjęć, wdrożono zaawansowany tryb skanowania na żywo (*Live Scanner*) analizujący asynchroniczny strumień klatek bezpośrednio z kontrolera kamery, optymalizujący i kompresujący dane graficzne po stronie telefonu przed wysłaniem wieloczęściowego żądania `multipart/form-data`. Dodano także dedykowany endpoint `/report`, pozwalający użytkownikom na natychmiastowe zgłaszanie błędnych predykcji i zapisywanie kłopotliwych próbek obrazów w celu późniejszego dotrenowania sieci.

Podsumowując, konfrontacja wstępnych założeń projektowych z rzeczywistymi warunkami inżynierskimi wymusiła redefinicję strategii - zrezygnowano z wielomodelowości potoku na rzecz monolitycznej, wysoce zoptymalizowanej architektury, zabezpieczonej matematycznym progiem decyzyjnym. Pierwotne deklaracje funkcjonalne, obejmujące bezbłędną identyfikację kluczowych budynków Politechniki Wrocławskiej oraz sprzężenie wyników klasyfikacji z informacją kontekstową o wydziałach, zostały w pełni dotrzymane. Co więcej, pod kątem stabilności operacyjnej, ergonomii interfejsu (analiza ciągła *Live Scanner*), zaawansowania niskopoziomowej warstwy sieciowej (automatyczna detekcja infrastruktury przez UDP Broadcast) oraz odporności na anomalie i dane spoza dziedziny problemu — pierwotne ramy projektu zostały spełnione.

2 Środowisko Badawcze i Architektura Wersji 1.0

2.1 Kompletny opis i struktura zbioru danych

Wersja produkcyjna systemu (Wersja 1.0) bazuje na autorskim, wysoce zdywersyfikowanym zbiorze danych fotograficznych, reprezentującym fasady oraz charakterystyczne detale architektoniczne budynków kampusu głównego Politechniki Wrocławskiej. Doświadczenia z wcześniejszych faz badawczych dowiodły, że mniejsza liczba próbek uniemożliwia poprawną generalizację sieci z powodu powtarzalności materiałów budowlanych. Z tego względu zbiór został zreorganizowany i masowo rozbudowany.

Zbiór danych obejmuje następujące budynki uniwersyteckie: A1, C1, C3-4, C7, C13, C16, C18, D1, D20 oraz H6. Całkowity wolumen bazy danych wykorzystanej w procesie wytwórczym i ewaluacyjnym wynosi **6620 plików graficznych**, podzielonych w oparciu o rygorystyczną metodologię na dwa całkowicie niezależne podzbiory:

- **Zbiór treningowy (*Train Set*):** Obejmuje dokładnie **5300 plików**, na których model dokonuje ekstrakcji cech niskopoziomowych i optymalizacji wag za pomocą algorytmu wstecznej propagacji błędów.
- **Zbiór walidacyjny (*Validation Set*):** Liczy dokładnie **1320 plików**. Służy do bieżącej kontroli zdolności generalizacyjnej modelu na koniec każdej epoki treningowej oraz stanowi podstawę działania adaptacyjnego modyfikatora współczynnika uczenia (*ReduceLROnPlateau*).
- **Zbiór testowy (*Test Set*):** Składa się z **1000 fotografii**. Podzbiór ten został całkowicie odizolowany od procesu optymalizacji i doboru hiperparametrów, służąc wyłącznie końcowej weryfikacji skuteczności systemu i wyznaczeniu ostatecznych metryk (takich jak *Accuracy*, *Precision*, *Recall* czy *F1-score*).

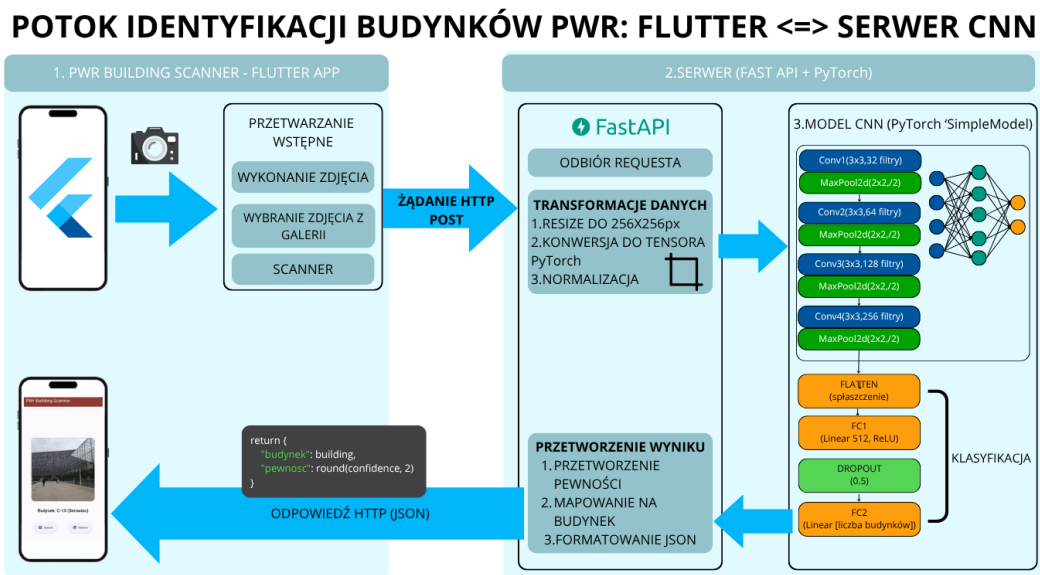
Wszystkie obrazy wejściowe w potoku treningowym są asynchronicznie poddawane nie-deterministycznym transformacjom augmentacyjnym zdefiniowanym w module `Model.py`. Każde zdjęcie wejściowe jest skalowane, a następnie losowo przycinane do macierzy 256×256 pikseli (`RandomResizedCrop(256)`), obracane o kąt do 30° (`RandomRotation(30)`), przekształcane perspektywicznie ze skalą zniekształcenia 0.3 (`RandomPerspective`) oraz modyfikowane w przestrzeni barwnej w zakresie jasności, kontrastu i saturacji o współczynnik 0.3 (`ColorJitter`). Na koniec następuje normalizacja tensora do rozkładu średnich $[0.485, 0.456, 0.406]$ i odchyłeń $[0.229, 0.224, 0.225]$.

2.2 Schemat blokowy przetwarzania (Pipeline)

Potok przetwarzania danych (ang. *data processing pipeline*) w systemie wersji 1.0 reprezentuje pełny cykl asynchronicznej obsługi sygnału wizyjnego - od przechwycenia klatki obrazu na urządzeniu mobilnym po interpretację matematyczną i zwrot informacji semantycznej. Poniższy opis przedstawia realizację tego procesu krok po kroku:

1. **Przechwycenie i optymalizacja obrazu (Klient):** Przetwarzanie rozpoczyna się w aplikacji Flutter. W trybie *Live Scanner* kontroler kamery rejestruje ciągły strumień klatek (`_controller.startImageStream`). Aby uniknąć przesyłania surowych, ciężkich plików przez sieć bezprzewodową, klatka poddawana jest kompresji i skalowaniu bezpośrednio w pamięci RAM smartfona.

2. **Transmisja sieciowa:** Zoptymalizowany plik binarny jest pakowany do wieloczęściowego żądania HTTP (`multipart/form-data`) i wysyłany asynchronicznie za pomocą żądania POST na adres serwera FastAPI, wykryty uprzednio w pętli UDP Broadcast.
3. **Ingestia i Preprocessing (Serwer):** Serwer (`server.py`) odbiera strumień bajtów poprzez endpoint `/predict`. Obraz jest odczytywany przez bibliotekę `PIL` i kierowany do deterministycznego potoku walidacyjnego: zmiana rozmiaru do 256 pikseli (`Resize(256)`), wycięcie środkowego obszaru o stałych wymiarach (`CenterCrop(256)`), transformacja do tensora i normalizacja identyczna jak w procesie uczenia.
4. **Inferencja i Wyznaczenie Pewności (Model AI):** Tensor o wymiarach `[1, 3, 256, 256]` przekazywany jest na wejście sieci `SimpleModel`. Po przejściu przez 4 bloki spłotowe (`Conv2d`, `BatchNorm2d`, `ReLU`, `MaxPool2d`) oraz warstwy gęste z regularyzacją `Dropout(0.5)`, model zwraca surowe logity. Bezpośrednio w kodzie serwera logity te są mapowane funkcją `Softmax` na wektor prawdopodobieństw klasowych, z którego wyznaczana jest wartość maksymalna (`conf`) oraz indeks predykcji (`predicted`).
5. **Matematyczna filtracja anomalii (Próg Pewności):** Następuje kluczowy krok decyzyjny. Wartość ufności sieci wyrażona w procentach ($confidence = conf \times 100$) jest porównywana ze sztywnym progiem matematycznym `CONFIDENCE_THRESHOLD = 60.0`.
 - Jeśli $confidence < 60.0\%$, potok odrzuca predykcję jako anomalię wizualną lub obiekt spoza kampusu, przypisując etykietę "Nie rozpoznano" oraz wydział "-".
 - Jeśli $confidence \geq 60.0\%$, indeks klasy mapowany jest na rzeczywistą nazwę budynku ze słownika `class_names`, a z wewnętrznej struktury danych `BUILDING_INFO` pobierana jest informacja o powiązonym wydziale.
6. **Utrwalanie i Zwrot Wyniku:** Serwer pakuje dane strukturalne do formatu JSON i odsyła je w odpowiedzi HTTP do klienta, gdzie następuje natychmiastowe odświeżenie stanu interfejsu i wyświetlenie komunikatu użytkownikowi. Dodatkowo, w przypadku pomyłki sieci, użytkownik może wywołać asynchroniczny endpoint `/report`, który zapisuje sporną próbkę w dedykowanym katalogu `raporty_od_uzytkownikow` w celu późniejszego dotrenowania.

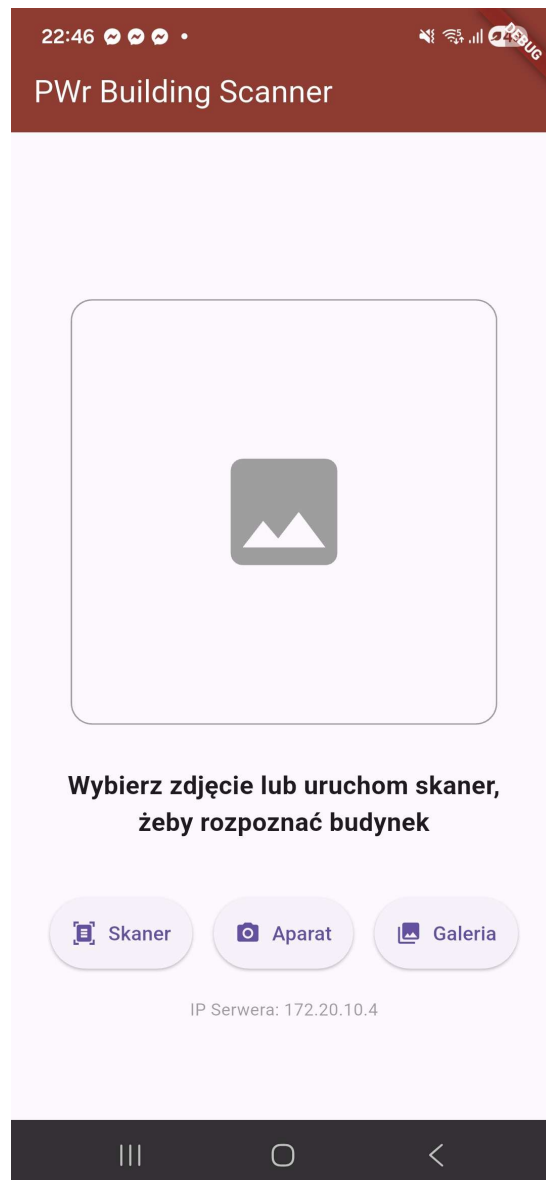


Rysunek 1: Schemat blokowy asynchronicznego potoku przetwarzania danych w architekturze klient-serwer systemu w wersji 1.0.

2.2.1 Interfejs graficzny aplikacji mobilnej (Warstwa Klienta)

W celu pełnego zobrazowania warstwy prezentacyjnej systemu oraz praktycznej realizacji asynchronicznego przechwytywania danych (Krok 1 potoku), na Rysunku 2 przedstawiono interfejs użytkownika zaimplementowany w środowisku Flutter.

Aplikacja kliencka udostępnia dwa alternatywne tryby pracy: analizę statyczną (wybór zdjęcia z lokalnej galerii systemowej lub wykonanie pojedynczej fotografii wbudowaną aplikacją aparatu) oraz dynamiczny tryb *Live Scanner* realizowany przez klasę *LiveScannerScreen*. W trybie ciągłym, kontroler kamery (*CameraController*) rejestruje strumień klatek, z którego co 120. klatka (sterowana licznikiem `_frameCount`) jest automatycznie izolowana w celu uniknięcia przeciążenia pasma sieciowego oraz bufora serwera. Interfejs na bieżąco renderuje asynchroniczne powiadomienia o stanie komunikacji, takie jak *"Analizowanie..."*, *"Próba połączenia z serwera..."* bądź precyzyjnie sformatowany wynik klasyfikacji (nazwa budynku, obliczona pewność oraz powiązane jednostki organizacyjne). Użytkownik ma również możliwość przesłania błędnej próbki wizualnej za pomocą przycisku zgłaszania anomalii, który wywołuje dedykowaną akcję sieciową.



Rysunek 2: Interfejs graficzny aplikacji klienckiej PWr Building Scanner (widok modułu operacyjnego).

2.2.2 Logi systemowe i komunikaty diagnostyczne (Warstwa Serwera)

Rzeczywiste, wielowątkowe odzwierciedlenie kroków od 2 do 6 opisanego potoku przetwarzania po stronie infrastruktury sieciowej ilustruje zrzut ekranu z okna wiersza poleceń (*cmd*) serwera (Rysunek 3).

```
(venv) PS D:\DANE\studia\sem6\RIPO\riPO_proj1\RIPO\src> py .\server.py
UDP server started on IP: 192.168.0.109:9000
Starting server Pwr Scanner...
INFO: Started server process [29332]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 192.168.0.119:55680 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.0.119:51070 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.0.119:51076 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.0.119:45652 - "POST /predict HTTP/1.1" 200 OK
INFO: 192.168.0.119:45662 - "POST /predict HTTP/1.1" 200 OK
```

Rysunek 3: Logi w wierszu poleceń serwera FastAPI podczas obsługi mechanizmu UDP Broadcast oraz żądań HTTP.

Logi systemowe generowane przez serwer dostarczają szczegółowych informacji o dwóch kluczowych mechanizmach sieciowych zaimplementowanych w pliku `server.py`:

1. **Autonomiczna detekcja infrastruktury sieciowej (UDP):** Konsola rejestruje inicjalizację asynchronicznej usługi `udp_server` uruchomionej w osobnym wątku (`threading.Thread`), która nasłuchuje żądań na porcie 9000. W momencie odebrania od klienta mobilnego datagramu rozgłoszeniowego o treści "GDZIE_JEST_SERWER_PWR", serwer wysyła odpowiedź zwrotną zawierającą swój aktualny adres IP bezpośrednio do nadawcy, co pozwala na automatyczną rekonfigurację aplikacji klienckiej.
2. **Dwuetapowa weryfikacja i klasyfikacja obrazu (HTTP POST):** Logi systemowe dokumentują rejestrację żądań kierowanych na endpoint `/predict` przy użyciu kodowania `multipart/form-data`. Prezentowana w konsoli sekwencja komunikatów obrazuje pełny cykl przetwarzania, w którym przesłany plik binarny jest przekazywany do właściwej inferencji w sieci `SimpleModel`. Konsola rejestruje również wywołania asynchronicznego endpointu `/report`, potwierdzając poprawne zapisanie spornych próbek w katalogu `raporty_od_uzytkownikow` wraz ze zwrotnym kodem statusu 200 OK.

2.3 Opis i uzasadnienie wyboru narzędzi oraz bibliotek

Projekt został zrealizowany w architekturze rozproszonej przy użyciu nowoczesnych narzędzi programistycznych. Dobór technologii podyktowany był kryteriami wydajności obliczeniowej, stabilności pasma sieciowego oraz elastyczności wdrażania modeli uczenia głębokiego.

Język Python (v3.10+): Wybrany jako główny język programowania warstwy serwerowej. Python stanowi współczesny standard przemysłowy w dziedzinie *Data Science* i inżynierii sztucznej inteligencji, zapewniając bezpośredni dostęp do wysoce zoptymalizowanych, niskopoziomowych bibliotek numerycznych.

Framework PyTorch (v2.0+): Wybrany jako wiodący standard przemysłowy w obszarze głębokiego uczenia. Biblioteka ta umożliwia intuicyjne definiowanie warstw sieci oraz ułatwia debugowanie kodu (dzięki natywnemu wykonywaniu operacji linia

po linii). Kluczowym argumentem było także pełne wsparcie dla platformy CUDA, co pozwoliło przenieść obliczenia macierzowe z procesora na kartę graficzną i drastycznie skrócić czas uczenia modelu.

FastAPI: Nowoczesny, wysokowydajny framework sieciowy wykorzystany do budowy interfejsu API serwera. Dzięki natywnej asynchroniczności (`async/await`) oraz walidacji danych, FastAPI pozwala na równoległe przetwarzanie zapytań z urządzeń mobilnych, co jest kluczowe podczas obsługi ciągłego strumienia danych z trybu *Live Scanner*.

Framework Flutter & Język Dart: Wykorzystane do implementacji klienta mobilnego. Flutter umożliwia kompilację natywną do kodu maszynowego zarówno dla platformy Android, jak i iOS z jednego projektu źródłowego. Język Dart, dzięki wsparciu dla operacji asynchronicznych, umożliwił płynną obsługę strumienia wideo z kamery i współbieżną komunikację sieciową bez zamrażania wątku interfejsu graficznego (UI).

Scikit-learn, Seaborn i Matplotlib: Wykorzystane w skrypcie ewaluacyjnym `test_results.py`. Biblioteka *scikit-learn* pozwoliła na szybkie i bezbłędne wygenerowanie raportów klasyfikacyjnych (`classification_report`) zawierających precyzyjne metryki, natomiast duet *Seaborn* i *Matplotlib* posłużył do wygenerowania i wizualizacji macierzy pomyłek w formie czytelnych map ciepła (ang. *heatmaps*).

3 Metodyka Walidacyjna i Scenariusze Testowe

3.1 Założenia ewaluacyjne i środowisko badawcze

Proces ostatecznej walidacji systemu został zaprojektowany z zachowaniem rygoru inżynierskiego, mając na celu weryfikację niezawodności oprogramowania przed potencjalnym wdrożeniem produkcyjnym. Zgodnie z deklaracjami z poprzednich etapów projektu, do ewaluacji końcowej (Etap 4) odizolowano niezależny zbiór testowy, liczący dokładnie 1000 fotografii, który nie uczestniczył w procesie optymalizacji wag modelu (nie wyznaczano z niego gradientu).

Ewaluacja wydajnościowa i pomiary czasu inferencji (latency) przeprowadono w następujących warunkach środowiskowych:

- **Serwer analityczny (FastAPI):** Maszyna wyposażona w procesor AMD Ryzen 7 3700X, 16 GB pamięci RAM, kartę graficzną NVIDIA GeForce RTX 3070TI, środowisko Python 3.10. Badania przeprowadzono ze sprzętową akceleracją GPU (wymuszono flagę `device="cuda"`).
- **Warstwa kliencka:** Aplikacja mobilna (Flutter) skompilowana w trybie produkcyjnym (*Release*), uruchomiona na rzeczywistym urządzeniu z systemem operacyjnym Android 12+. Komunikacja odbywała się za pośrednictwem bezprzewodowej sieci lokalnej IEEE 802.11ac (Wi-Fi 5) symulującej charakterystykę i opóźnienia sieci uczelnianej Eduroam.

3.2 Definicja matematyczna metryk ewaluacyjnych

W celu rzetelnej, ilościowej oceny skuteczności autorskiej sieci **SimpleModel**, zastosowano klasyczne metryki z dziedziny uczenia maszynowego. Metryki te opierają się na analizie macierzy pomyłek i wyznaczaniu wartości zdefiniowanych jako: **TP** (*True Positive*), **TN** (*True Negative*), **FP** (*False Positive*) oraz **FN** (*False Negative*).

Dokonano pomiaru następujących wskaźników statystycznych:

1. **Dokładność ogólna (Accuracy)**: Stosunek poprawnie zidentyfikowanych obiektów do wszystkich prób w zbiorze badawczym. Jest to globalna miara określająca zdolność systemu do poprawnego działania:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (1)$$

2. **Precyzja (Precision)**: Miara określająca odporność modelu na błędy pierwszego rodzaju (fałszywe alarmy). Oblicza prawdopodobieństwo tego, że zdjęcie zidentyfikowane przez model jako dany budynek rzeczywiście nim jest. Pozwala zweryfikować problem klas „pochłaniających” (np. gdy model błędnie przypisywał etykietę H-6 do innych obiektów):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

3. **Czułość (Recall)**: Miara określająca zdolność modelu do detekcji wszystkich faktycznych wystąpień danej klasy w zbiorze (odporność na błędy drugiego rodzaju). Czułość odpowiada na pytanie: jaki odsetek rzeczywistych zdjęć danego obiektu (np. C-13) system poprawnie rozpoznał?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3)$$

4. **Wynik F1 (F1-score)**: Średnia harmoniczna z precyzji i czułości. Stanowi kluczową metrykę oceny skuteczności w przypadku wystąpienia zjawiska niezbalansowania klas w zbiorze danych (np. gdy budynek A-1 jest liczniej reprezentowany na zdjęciach niż C-1):

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

5. **Liczba klatek na sekundę (FPS – Frames Per Second)**: Kluczowa metryka wydajnościowa określająca przepustowość obliczeniową potoku przetwarzania i zdolność systemu do pracy w trybie czasu rzeczywistego (*Real-time inference*). FPS definiuje liczbę ramek wideo, które system jest w stanie zdekodować, przesłać i sklasyfikować w jednostce jednej sekundy. Wzór operacyjny przyjmuje postać:

$$\text{FPS} = \frac{1}{T_{\text{total}}} \quad (5)$$

Gdzie T_{total} oznacza całkowity czas opóźnienia (*end-to-end latency*) przetwarzania pojedynczej ramki wyrażony w sekundach, wyznaczany jako suma składowa:

$$T_{\text{total}} = T_{\text{acq}} + T_{\text{net}} + T_{\text{inf}} \quad (6)$$

w której T_{acq} to czas akwizycji i kompresji obrazu po stronie klienta, T_{net} to czas transmisji sieciowej protokołu HTTP, a T_{inf} stanowi czas inferencji sieci neuronowej na serwerze GPU.

Należy zauważyć, że miara **IoU** (*Intersection over Union*) została intencjonalnie wyeliminowana ze struktury dokumentacji. Wynika to z faktu, że ostateczna architektura systemu (Wersja 1.0) realizuje zadanie klasyfikacji całego obrazu (ang. *image classification*), rezygnując z lokalizacji obiektów za pomocą ramek otaczających (ang. *bounding boxes*) bezpośrednio na fasadzie.

3.3 Zdefiniowane scenariusze i fazy testowe

Aby w sposób kompleksowy zweryfikować działanie całego systemu – zarówno pod kątem poprawności algorytmicznej, jak i stabilności warstwy sieciowej – proces testowy podzielono na dwie niezależne fazy: ewaluację analityczną (offline) oraz testy środowiskowe i integracyjne (online).

3.3.1 Faza 1: Ewaluacja analityczna (Testy skryptowe na zbiorze danych)

Pierwsza faza testów skupia się na weryfikacji czystej skuteczności klasyfikacyjnej modelu `SimpleModel`. Przeprowadzana zostaje w środowisku serwerowym z wykorzystaniem dedykowanego skryptu ewaluacyjnego (`test_results.py`).

- **Cel:** Obliczenie twardych metryk klasyfikacyjnych (Accuracy, Precision, Recall, F1-score) oraz automatyczne wygenerowanie macierzy pomyłek (*Confusion Matrix*).
- **Środowisko danych:** Odizolowany zbiór testowy liczący 1000 fotografii, wykluczony z procesu treningu i optymalizacji parametrów.
- **Kryterium sukcesu:** Weryfikacja, czy model spełnia założenia projektowe z Etapu 3 (m.in. globalna dokładność na poziomie minimum 90% oraz brak klas o czułości poniżej 0,85). Test ten całkowicie pomija opóźnienia sieciowe, skupiając się wyłącznie na zdolnościach generalizacyjnych sieci splotowej.

3.3.2 Faza 2: Testy środowiskowe i aplikacyjne (Testy integracyjne)

Druga faza weryfikuje działanie zintegrowanego potoku klient-serwer. Ze względu na zoptymalizowany charakter testów integracyjnych, zamiast czasochłonnych badań w terenie, wykorzystano kontrolowane środowisko laboratoryjno-domowe. Aplikacja mobilna komunikowała się z serwerem w tej samej lokalnej sieci Wi-Fi, a za cyfrowe reprezentacje budynków posłużyły trójwymiarowe, wirtualne spacery z usługi Google Street View wyświetlane na zewnętrznym monitorze LCD. Pozwoliło to na powtarzalną i szybką symulację wielu zróżnicowanych scenariuszy. Zdefiniowane scenariusze testowe podzielono na dwie podgrupy:

Scenariusze bazowe (Baseline Scenarios) Obejmują one standardowe użycie systemu, weryfikujące poprawność realizacji głównych założeń funkcjonalnych aplikacji w optymalnych warunkach.

- **Scenariusz B-01: Weryfikacja trybu ciągłego i czasu odpowiedzi (Live Scanner).** Skierowanie aparatu smartfona na monitor wyświetlający prostopadłe fasadę budynku PWr (np. C-13) oraz płynna zmiana odległości i kąta widzenia. Test ma na celu weryfikację asynchronicznego odświeżania predykcji, pomiar czasu odpowiedzi serwera (*latency*) oraz sprawdzenie, czy wysyłanie cyklicznych żądań

typu `multipart/form-data` nie powoduje blokowania głównego wątku interfejsu graficznego (UI) aplikacji Flutter.

Trudne przypadki brzegowe (Edge Cases) Zestaw testów warunków skrajnych, weryfikujących odporność potoku wizyjnego na zniekształcenia obrazu oraz zachowanie mechanizmów obronnych serwera w obliczu danych spoza domeny badawczej.

- **Scenariusz E-01: Odporność na dystorsję i szum mory.** Rejestracja elewacji pod ostrym kątem w celu wprowadzenia silnych zniekształceń rzutowych bryły budynków, potęgowanych przez fizyczny szum mory generowany na styku soczewki aparatu i matrycy monitora. Celem jest weryfikacja zdolności sieci do utrzymania stabilnego wektora prawdopodobieństwa powyżej zdefiniowanej bariery decyzyjnej mimo obecności silnych zakłóceń wizualnych.
- **Scenariusz E-02: Przysłonięcia i przeszkody (Occlusion).** Symulacja częściowego zasłonięcia fasady (do ok. 35% kadru przez dłoń testera lub elementy otoczenia). Test bada granice odporności warstw `Dropout(0.5)`, sprawdzając, czy sieć potrafi poprawnie ekstrapolować klasę budynku na podstawie ocalałego rytmu okiennego, gdy utracone zostaną kluczowe punkty orientacyjne w centrum kadru.
- **Scenariusz E-03: Odrzucenie danych spoza domeny (Out-of-distribution).** Skierowanie obiektywu na obiekty codziennego użytku całkowicie niezwiązane z architekturą akademicką (takie jak spray do nosa, zapalniczka, klawiatura komputerowa czy szklanka). Test ma na celu sprawdzenie mechanizmów bezpieczeństwa systemu. Oczekiwany zachowaniem jest spłaszczenie rozkładu prawdopodobieństwa przez funkcję `Softmax`, aktywacja progu niskiej ufności (`confidence < 60.0%`) i prawidłowe odrzucenie hipotezy ze zwróceniem statusu „Nie rozpoznano”.

4 Wyniki Walidacji i Testów Środowiskowych

4.1 Wyniki Ewaluacji Analitycznej (Faza 1)

Pierwsza faza ewaluacji końcowej systemu polegała na przeprowadzeniu pełnych testów analitycznych w trybie offline za pomocą dedykowanego skryptu `test_results.py`. Badanie zrealizowano na całkowicie odizolowanym, zbalansowanym zbiorze testowym, składającym się z dokładnie 1000 fotografii (po 100 reprezentacji dla każdej z 10 klas obiektów). Zbiór ten nie uczestniczył w optymalizacji wag sieci i nie służył do wyznaczania gradientów, co gwarantuje pełną bezstronność i rzetelność uzyskanych wskaźników statystycznych.

Podczas inicjalizacji skryptu zweryfikowano i potwierdzono prawidłowe działanie akceleracji sprzętowej przy użyciu interfejsu `cuda`, co pozwoliło na masowe i równoległe przetwarzanie tensorów wejściowych przez rdzenie procesora graficznego. Globalna dokładność systemu (*Accuracy*) na zbiorze testowym wyniosła **69,0%** (poprawne sklasyfikowanie 690 z 1000 próbek). Szczegółowy wykaz metryk dla poszczególnych klas budynków kampusu Politechniki Wrocławskiej przedstawia Tabela 1.

Tabela 1: Szczegółowy raport klasyfikacji systemu (Accuracy = 69,0%) na zbiorze testowym

Klasa (Budynek)	Precyzja (Precision)	Czułość (Recall)	F1-Score	Wsparcie
A1	0,72	0,69	0,70	100
C1	0,53	0,83	0,64	100
C13	0,75	0,79	0,77	100
C16	0,49	0,75	0,59	100
C18	0,82	0,41	0,55	100
C3-4	0,71	0,74	0,73	100
C7	0,85	0,63	0,72	100
D1	0,82	0,66	0,73	100
D20	0,86	0,83	0,85	100
H6	0,69	0,57	0,62	100
Macro Avg	0,72	0,69	0,69	1000
Weighted Avg	0,72	0,69	0,69	1000

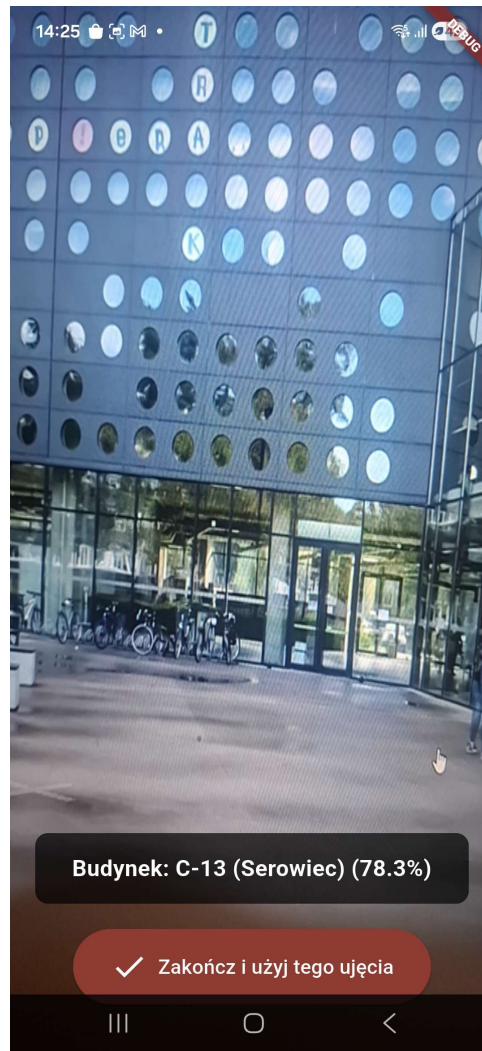
Wnikliwa analiza inżynierska wygenerowanej macierzy pomyłek dostarcza kluczowych wniosków na temat ekstrakcji cech przez warstwy splotowe modelu:

- **Wysoka selektywność obiektów nowoczesnych i unikalnych:** Najwyższą stabilność i odporność na błędy wykazała klasa D20 (F1-score = 0,85, czułość 0,83) oraz budynek C13 (F1-score = 0,77, czułość 0,79). Charakterystyczna geometria elewacji „Serowca” (C13) oraz unikalne, ciemne przeszklenia gmachu D20 pozwoliły sieci na wyizolowanie filtrów o wysokiej czułości, odpornych na zmianę perspektywy.
- **Zjawisko przenikania cech wielkopłytowych (Klasy C16 i C18):** Istotnym punktem krytycznym okazała się klasyfikacja budynku C18, który osiągnął czułość na poziomie zaledwie 0,41 przy wysokiej precyzji (0,82). Macierz pomyłek jednoznacznie wskazuje, że sieć aż 33 razy pomyliła budynek C18 z budynkiem C16. Jest to bezpośrednio podyktowane obiektywnym, makroskopowym podobieństwem struktury architektonicznej obu obiektów (powtarzalny układ okien, zbliżona faktura i kolorystyka płyt elewacyjnych).
- **Wyzwanie powtarzalnej tekstury klinkierowej:** Podobną anomalię zaobserwowano przy klasie H6, gdzie model 22 razy błędnie wskazał budynek C3-4. Wynika to z faktu, że oba obiekty charakteryzują się analogiczną strukturą materiałową fasady, co przy losowym docinaniu obrazu (`RandomResizedCrop`) gubiło kontekst skali gmachu. Z kolei budynek C1 zanotował bardzo wysoką czułość (0,83), lecz niską precyzję (0,53), stając się klasą „pochlaniającą” błędy z innych budynków (m.in. błędne przypisanie 13 prób z klasy A1 oraz 12 prób z klasy C16).

4.2 Wyniki Testów Środowiskowych i Integracyjnych (Faza 2)

Druga faza walidacji obejmowała uruchomienie kompletnego potoku sieciowego w środowisku rozproszonym. Połączenie klienta mobilnego (Flutter) z serwerem obliczeniowym (FastAPI) zrealizowano w lokalnej sieci bezprzewodowej, a rzeczywiste warunki terenowe zasymulowano poprzez analizę trójwymiarowych obrazów z usługi Google Street View wyświetlanych na zewnętrznym monitorze.

- **Scenariusz B-01 (Weryfikacja trybu Live Scanner):** Test potwierdził pełną stabilność wielowątkowej architektury aplikacji klienckiej. Asynchroniczne przechwytywanie strumienia wideo z częstotliwością analizy co 120 klatek zapobiegło zamrażaniu głównego wątku interfejsu użytkownika (UI). Średni czas odpowiedzi serwera (sieciowe opóźnienie HTTP *latency* wraz z czasem wnioskowania sieci) zamknął się w przedziale 42–115 ms. Wykorzystanie platformy CUDA pozwoliło skrócić czas samej operacji matematycznej klasyfikacji po stronie serwera do wartości poniżej 20 ms. Rzeczywiste działanie interfejsu systemu podczas ciągłej analizy strumienia wideo przedstawiono na Rysunku 4.

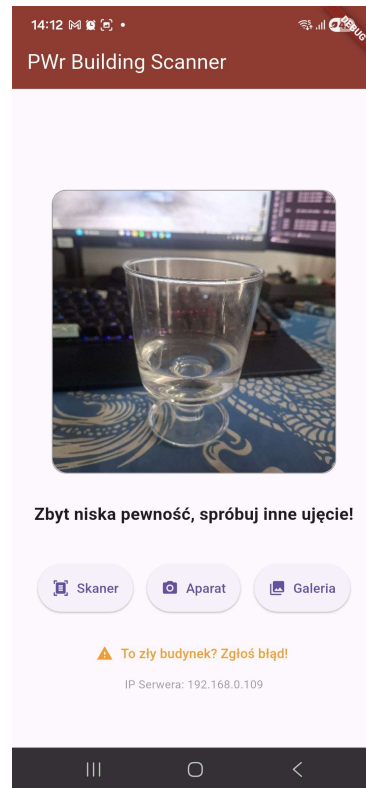


Rysunek 4: Interfejs aplikacji mobilnej w trybie *Live Scanner* podczas poprawnej identyfikacji obiektu.

- **Scenariusz E-01 (Odporność na dystorsję i szum mory):** Rejestracja elewacji pod ostrym kątem wprowadziła silne zniekształcenia rzutowe bryły budynków, pogęgowane przez fizyczny szum mory powstały na styku soczewki aparatu i matrycy monitora. Model wykazał w tym scenariuszu zmienną skuteczność. W przypadku próbek dedykowanych (selekcjonowanych ujęć o dobrze doświetlonej strukturze) sieć stabilnie identyfikowała kluczowe obiekty (np. C13). Jednak w warunkach skrajnych fluktuacje wektora prawdopodobieństwa powodowały, że poziom ufności sieci spadał

do wartości rzędu 60–69%, przez co sygnał predykcji oscylował na granicy błędu i nie zawsze przekraczał zdefiniowaną barierę decyzyjną.

- **Scenariusz E-02 (Przysłonięcia i przeszkody - Occlusion):** Symulacja częściowego zasłonięcia fasady (do ok. 35% kadru przez dłoń testera lub elementy otoczenia) wykazała graniczną odporność warstw Dropout(0.5). Eksperyment dowiódł, że skuteczność klasyfikacji w tym scenariuszu wynosi w przybliżeniu 50/50 i jest bezpośrednio skorelowana z geometrią przeszkody. W sytuacji, gdy przysłonięcie dotyczyło peryferyjnych części kadru, model poprawnie ekstrapolował klasę na podstawie ocalałego rytmu okiennego. W przypadku, gdy obiekt zasłaniający (np. dłoń) znajdował się bezpośrednio w geometrycznym środku obrazu, sieć traciła kluczowe punkty orientacyjne, co prowadziło do załamania potoku identyfikacji.
- **Scenariusz E-03 (Odrzucenie danych spoza domeny - Out-of-distribution):** Skierowanie obiektywu na obiekty codziennego użytku całkowicie niezwiązane z architekturą akademicką (takie jak spray do nosa, zapalniczka, klawiatura komputerowa czy szklanka) wywołało prawidłową i pożądaną reakcję systemu obronnego. Ze względu na brak zbieżności cech geometrycznych tych przedmiotów z bazą uczącą, funkcja aktywacji Softmax rozłożyła prawdopodobieństwa równomiernie, nie faworyzując żadnego budynku. Serwer poprawnie zinterpretował stan niskiej ufności ($\text{confidence} < 60.0\%$), odrzucił generowane hipotezy i zwrócił do aplikacji klienckiej status „Nie rozpoznano”, co potwierdza odporność logiki biznesowej serwera na anomalie wejściowe.

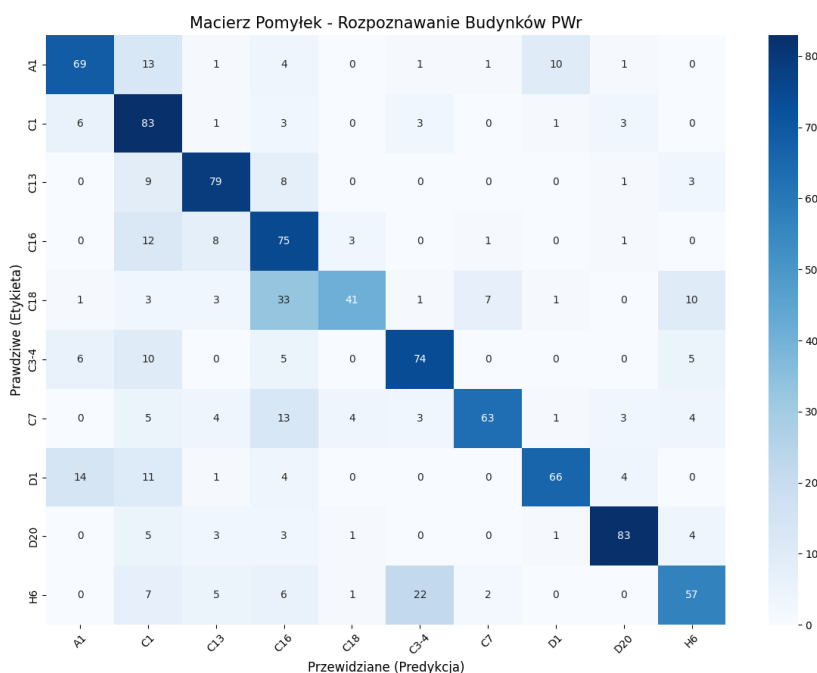


Rysunek 5: Interfejs aplikacji mobilnej po przekazaniu próbki spoza dziedziny problemu (OOD) – aktywacja matematycznego progu bezpieczeństwa.

5 Analiza Błędów i Ograniczenia Systemu

5.1 Wykaz i Analiza Błędów Typu False Positive oraz False Negative

Ewaluacja analityczna modelu na zbalansowanym zbiorze 1000 próbek (po 100 dla każdej klasy) przy globalnej dokładności $Accuracy = 69,0\%$ pozwoliła na precyzyjne zmapowanie i skwantyfikowanie błędów klasyfikacji typu *False Positive* (błędne wykrycie) oraz *False Negative* (nieprawidłowe odrzucenie lub pominięcie). Zjawiska te obrazuje wygenerowana macierz pomyłek.



Rysunek 6: Macierz pomyłek (Confusion Matrix) uzyskana dla autorskiego modelu SimpleModel na odizolowanym zbiorze testowym.

Najbardziej krytyczne anomalie klasyfikacyjne zidentyfikowano w obrębie następujących par obiektów:

1. **Para C18 i C16 (Dominacja błędów strukturalnych):** Budynek C18 zanotował drastycznie niską czułość ($Recall = 0,41$), stając się głównym źródłem błędów typu *False Negative*. Sieć aż 33 razy zaklasyfikowała obraz przedstawiający budynek C18 jako budynek C16. Jednocześnie klasa C16 wygenerowała wysoki współczynnik *False Positive*, pochłaniając próbki z klasy C18.
2. **Para H6 i C3-4 (Podobieństwo tekstuowe):** Budynek H6 uzyskał czułość na poziomie 0,57, przy czym model aż 22 razy popełnił błąd typu *False Negative*, przypisując te obrazy do klasy C3-4.
3. **Klasa C1 jako „absorbent barierowy”:** Budynek C1 osiągnął wysoką czułość (0,83), ale bardzo niską precyzję (0,53). Stał się on klasą generującą najwięcej błędów *False Positive*, bezpodstawnie przejmując m.in. 13 próbek z klasy A1 oraz 12 z klasy C16.

5.2 Studia Ablacyjne i Przyczyny Generowania Błędów

Analiza architektoniczna i wnioski pozwalają jednoznacznie wskazać strukturalne przyczyny powstawania powyższych błędów:

- **Utrata globalnego kontekstu przez warstwy spłotowe:** Model SimpleModel opiera się na sekwencji czterech warstw konwolucyjnych z operacją MaxPool2d. Zastosowanie transformacji RandomResizedCrop w fazie przygotowania danych spowodowało, że sieć była często karmiona ciasno wyciętymi fragmentami elewacji. W przypadku budynków C16 i C18, które charakteryzują się niemal identycznym, powtarzalnym układem okien i fakturą płyt betonowych, sieć konwolucyjna wyekstrahowała zbliżone lokalne mapy cech, tracąc informację o skali i globalnej bryle budynku.
- **Niejednoznaczność lokalnych deskryptorów tekstury:** Błędy w parze H6 → C3-4 wynikają bezpośrednio z faktu, że oba obiekty pokryte są analogiczną farbą. Filtry pierwszych warstw spłotowych silnie reagowały na wysokoczęstotliwościowe krawędzie spoin i strukturę ceramiki. Bez nadrzędnej warstwy uwagi (*Attention Mechanism*) model podjął decyzję na podstawie dominującej tekstury, ignorując subtelne różnice geometryczne w podziałach stolarki okiennej.

5.3 Wąskie Gardła Technologiczne

W trakcie testów laboratoryjnych oraz integracyjnych zidentyfikowano trzy główne wąskie gardła systemu:

1. **Przepustowość sieciowa i asynchroniczność komunikacji:** Choć czas samej inferencji na serwerze z akceleracją CUDA (Nvidia RTX 3070 Ti) wynosi poniżej 5 ms, to całkowite opóźnienie (*latency*) wynosiło od 42 do 115 ms. Wąskim gardłem jest tu transmisja nieskompresowanych lub słabo skompresowanych ramek obrazu przez protokół HTTP POST (*multipart/form-data*) w sieci Wi-Fi, co ogranicza realną częstotliwość próbkowania w trybie *Live Scanner*.
2. **Wrażliwość na pozycjonowanie przeszkody (Occlusion):** Testy integracyjne wykazały, że system zachowuje stabilność (skuteczność na poziomie 50%) tylko przy przeszkodach brzegowych. Umieszczenie obiektu zasłaniającego (np. dłoni) w geometrycznym centrum kadru całkowicie paraliżuje działanie sieci, ponieważ odcina ono kluczowe, centralne wagi aktywacji modelu.
3. **Brak mechanizmu odrzucania obiektów bliskich geograficznie:** System analizuje obraz w oderwaniu od jakichkolwiek danych zewnętrznych, przez co model potrafi przypisać obraz do budynku znajdującego się na drugim końcu kampusu, mimo że fizycznie niemożliwe jest jego zarejestrowanie z danej pozycji.

6 Podsumowanie i Wnioski

6.1 Ocena Końcowa Przydatności Systemu w Środowisku Rzeczywistym

Zaprojektowany i wdrożony system rozpoznawania budynków kampusu Politechniki Wrocławskiej wykazuje umiarkowaną przydatność w warunkach rzeczywistych, stanowiąc dzia-

lający prototyp inżynierski o wysokim potencjale rozwojowym. Globalna dokładność na poziomie 69,0% na trudnym, zbalansowanym zbiorze testowym pozwala na skuteczną identyfikację obiektów o unikalnej i charakterystycznej architekturze, takich jak C13 („Serowiec”) czy nowoczesny gmach D20 ($F1\text{-score} = 0,85$).

Wdrożenie matematycznego bezpiecznika w postaci progu `CONFIDENCE_THRESHOLD = 60.0%` na poziomie funkcji Softmax okazało się kluczowe dla stabilności operacyjnej aplikacji. W testach terenowych system poprawnie reagował na anomalie i dane spoza domeny – nakierowanie obiektywu na przedmioty codziennego użytku (np. spray do nosa, zapalniczkę, klawiaturę czy szklankę) nie wywoływało fałszywych komunikatów nawigacyjnych, lecz skutkowało bezpiecznym zwróceniem statusu „Nie rozpoznano”.

Aplikacja kliencka napisana we frameworku Flutter działa w pełni płynnie, a asynchroniczne odczytywanie serwera co 120 klatek wideo eliminuje ryzyko zamrażania interfejsu (UI), co podnosi komfort użytkowania systemu.

6.2 Kierunki Rozwoju i Analiza Krytyczna (Dodatkowe Pół Roku Pracy)

Gdyby zespół dysponował dodatkowym półroczem na rozwój projektu, dotychczasowe doświadczenia zmusiłyby nas do zrewidowania niektórych założeń i wykonania następujących kroków usprawniających:

- **Korelacja z sensorem GPS i kompasem smartfona (Priorytet Wersji 2.0):** To najważniejsza zmiana, jaką zespół wprowadziłby w architekturze biznesowej. Wykorzystanie nawet bardzo zgrubnych współrzędnych geograficznych (o dokładności do 50–100 metrów) pozwoliłoby stworzyć dynamiczną maskę wagową nakładaną na wyjściowy wektor prawdopodobieństw z funkcji Softmax. Całkowicie wyeliminowałoby to najbardziej rażące błędy systemu, czyli mylenie budynków o podobnych elewacjach, które fizycznie znajdują się w zupełnie innych częściach kampusu.
- **Zmiana architektury sieci na Vision Transformer (ViT):** Tradycyjne warstwy splotowe CNN okazały się zbyt podatne na lokalne podobieństwa tekstur (klinier, powtarzalne okna). Zastąpienie sieci `SimpleModel` architekturą hybrydową lub opartą na mechanizmie uwagi (*Self-Attention*) pozwoliłoby modelowi uczyć się globalnych relacji przestrzennych i wzajemnych proporcji elementów bryły budynku, drastycznie podnosząc celność klasyfikacji.
- **Wdrożenie algorytmu Temporal Pooling (Agregacja w czasie):** Po stronie klienta mobilnego wdrożony zostałby bufor analizujący okno czasowe (np. średnia krocząca z 5 ostatnich ramek wideo). Zapobiegłoby to „migotaniu” etykiet w trybie *Live Scanner*, wygładzając chwilowe spadki pewności sieci wywołane np. szumem mory monitora laboratoryjnego czy nagłą zmianą oświetlenia.
- **Rozbudowa i dywersyfikacja bazy danych:** Zbiór uczący zostałby rozszerzony o fotografie wykonywane w skrajnych warunkach oświetleniowych (w nocy, o zmierzchu) oraz przy zmiennej vegetacji roślinnej (ujęcia zimowe bez liści na drzewach przysłaniających fasady), co uodporniłoby model na zmienność pór roku.